AirBnB Rating Prediction

DS in Production – Nebius Academy IL
Team: Ruth Adler, Ido Friedmann, Ella Yakir, Rosemary Lavender

---

1. Business Understanding (CRISP-DM Phase 1)

1.1 Problem statement

Airbnb hosts aim to maximize their listing ratings, which strongly affect demand and revenue. The goal of this project is to **predict the listing rating (review_scores_rating)** based on listing, host, and textual attributes, in a way that **generalizes across cities**.

1.2 Business objective

- Provide a predictive service that estimates expected ratings for listings
- Enable hosts to understand which controllable factors influence ratings
- Ensure the model works **in any city**, not just those seen during training

1.3 Data science objective

- Build a regression model that improves over a naive baseline (mean prediction)
- Optimize **RMSE** while avoiding overfitting
- Emphasize generalization to unseen cities (Chicago test set)

1.4 Constraints & assumptions

- Ratings are highly compressed (most values between 4.5–5.0)
- No access to user-level review text
- Must avoid city-specific leakage
- Model should be deployable as a service

Airbnb hosts depend on high listing ratings, as these directly influence booking demand and revenue. The objective of this project was to predict the average overall listing rating (review_scores_rating column) using a combination of listing characteristics, host attributes, and available textual information from Los Angeles and New York City AirBnB datasets, in a way that generalized across different cities. From a data science perspective, we aim to build a regression model that outperforms a naive dummy RMSE, while avoiding overfitting. From a business perspective, the goal was to provide a predictive service that estimates expected ratings and may help hosts to understand which controllable factors most strongly affect guest satisfaction. A key requirement is robustness to unseen locations, evaluated via a held-out validation city (Chicago) and a final test set (Sydney) to ensure geographic

generalization. The analysis assumes a strict avoidance of city-specific data leakage, with the final model designed to be deployable in a real-world setting.

---

2. Data Understanding (CRISP-DM Phase 2)

2.1 Dataset description

- Source: Airbnb listings dataset
- Training data: Listings from **LA and NYC**
- Validation data: Listings from **Chicago**
- Test data: Listings from **Sydney**
- Target variable: review_scores_rating
- Feature groups:
    - Property attributes (beds, baths, accommodates)
    - Host attributes (superhost status, response rate, tenure)
    - Listing text fields

2.2 Target variable analysis

- Distribution heavily skewed toward high ratings
- ~95% of values fall between 4.5 and 5.0
- Strong **ceiling effect**, limiting achievable performance gains

2.3 Baseline performance

- Dummy regressor (mean prediction)
- RMSE ≈ 0.406
  This establishes a strong baseline and highlights the difficulty of the task.

2.4 Initial hypotheses

Examples tested:

- Superhosts receive higher ratings
- More bathrooms correlate with better ratings
- Hosts with many listings receive slightly worse ratings
- Price and location may be strong predictors

There were various groups that feature types could be gathered under; Property attributes included accommodates, bathrooms, bedrooms, beds, capturing the capacity of the listing. Occupancy and booking constraints are represented by features such as the minimum and maximum number of nights a guest can stay, and whether it was instantly bookable, reflecting availability rules. Host attributes and experience signals include whether they are a superhost, and their response rate, which proxy reliability and professionalism. Listing page

and textual indicators are captured through if there is a description, and an about page, summarizing the presence and richness of descriptive text without using raw text itself, and finally, the geographic aspects of the property were pinpointed with longitude and latitude.

Exploratory analysis of the target revealed a highly compressed distribution: approximately 95% of ratings fall between 4.5 and 5.0, indicating a strong ceiling effect. This lack of variance substantially constrains predictive performance, as even meaningful differences between listings translate into very small numerical changes, increasing sensitivity to noise and limiting achievable RMSE improvements. A dummy regressor predicting the mean rating achieved an RMSE of about 0.406, establishing a strong baseline and underscoring the difficulty of the task.

Correlations Analysis

We investigated associations between numeric variables and the target variable review_scores_rating.

- **Host experience:** Time as a host (time_as_host), which was a feature converted from date-time format, and occupancy over the past year (estimated_occupancy_l365d) showed a weak positive correlation with review scores (Pearson r ≈ 0.08-0.12 across cities).
- **Description length:** The number of words and characters in the property description type (description_length_words and description_length_chars), which was a feature converted from the textual format for ease of analysis illustrated a weak positive correlation across cities (both Pearson r ≈ 0.05-0.09).
- **Price:** The log-transformed price feature also showed a weak positive correlation to review scores (Pearson r ≈ 0.05-0.10).
- **Host load:** Log-transformed host listing counts (log_host_listings_count, log_host_total_listings_count) were weakly negatively correlated with review scores (r ≈ -0.17–-0.19). This indicates that hosts with more listings may slightly underperform in guest ratings.

The correlation heatmap below illustrates the correlation matrix comparing numeric parameters across LA, NY, and combined datasets. Overall, correlations are weak but reasonably equal over global, LA and NYC analyses, suggesting no single numeric feature dominates review scores.

Host attributes

Binary host attributes were analyzed using t-tests:

- Hosts with a profile picture (host_has_profile_pic) and an “About” section (has_host_about) had significantly higher ratings ($p < 0.001$).
- Superhosts, as expected, received higher review scores ($p < 0.001$).

Property and Listing attributes

Binary host attributes, including host_has_profile_pic, has_host_about, and Superhost status, were all significantly associated with higher ratings (t-test, $p < 0.001$). Property characteristics similarly influenced review scores: entire homes and unique listings generally received higher ratings than private or shared rooms (ANOVA with Tukey, $p < 0.001$), and additional bathrooms increased scores, while extra bedrooms or beds had smaller positive effects. Overall, these analyses highlighted host experience, property novelty, and profile completeness as key positive predictors of review ratings.

Hypotheses

From our initial statistical exploration of the data, and our knowledge of AirBnb, we created seven hypotheses. These were as follows:

1. Experience: More experienced hosts tend to have slightly higher ratings.
2. Demand: Higher estimated occupancy is associated with slightly higher ratings.
3. Superhost: Listings with superhost tend to have better ratings.
4. Number of listings: Hosts with more listings tend to have slightly lower ratings.
5. Entire homes and unique properties will receive higher ratings
6. Host profile completeness will increase ratings on the properties
7. Adding additional bathrooms and bedrooms will increase ratings

---

3. Data Preparation (CRISP-DM Phase 3)

3.1 Feature engineering

Initial feature pool included:

- Numeric listing attributes
- Host metadata
- Geographic features
- Text-based features
- Engineered distance-to-center feature

3.2 Feature selection mistakes (Distance Bug)

- City center computed across LA and NYC combined
- Resulting “center” located in central USA
- Distance feature became meaningless
- Lesson: **visualize features before trusting them**

3.3 Text feature approaches

- Tried embeddings and LLM-based extraction
- High cost, latency, and limited signal
- Final approach: **simple keyword indicators**
    - mentions_clean
    - mentions_luxury
    - mentions_modern

3.4 Final feature set

- Reduced to **22 simple, city-agnostic features**
- No geographic features
- No city-specific encodings
- Focus on interpretability and transferability

General data preparation + Preprocessing

The LA and NYC Airbnb datasets were loaded, containing 39 columns and 45,886 / 36,111 rows, respectively. Supplementary geographic datasets were also imported: NYC's borough boundaries (nynta2020.shp) and LA's Service Planning Areas (Service_Planning_Areas_2022.geojson). Each listing was assigned a corresponding area based on its geographic coordinates in a new column area, and a city column was added. The datasets were then concatenated into a single dataframe. Fifty duplicate rows were removed.

Columns were converted to appropriate data types:

- last_scraped, host_since, first_review, and last_review were converted to date time format.
- host_is_superhost, host_has_profile_pic, instant_bookable were converted to boolean objects
- host_response_rate, host_acceptance_rate, price were converted to numeric values
- host_verifications and amenities were parsed into Python lists

Listings missing the target variable were removed (accounting for 28.85% of entries), along with 194 rows missing multiple host attributes. Beds and bedrooms were imputed with the group median based on how many people the property accommodates. If the host was missing a value in the superhost status column, it was presumed this value would be False.

Feature Engineering

In addition to data imputations, several derived features were added to improve predictive signal. Binary indicators `has_host_about` and `has_neighborhood_overview` captured whether hosts included profile or neighborhood information. `Property_type_grouped` categorized listings into four groups: private room, shared room, entire home/apt, and unique (non-conventional) properties (including huts, cabins and caravans). Host response times were encoded numerically (`host_response_time_coded`: 1–4, with missing values as 0), and each listing was assigned an `area` based on geographic coordinates. Finally, listing description length (`desc_length`) was included as a simple numeric feature capturing text detail.

Feature Engineering - mistakes (the distance bug)

The distance-to-center feature was originally created to measure each listing's distance to its city center, but because the center was incorrectly computed across all training data from both LA and NYC, it fell somewhere in central USA (Kansas) and was meaningless,

illustrating the importance of visualizing features before trusting them. The initial distance-to-center feature highlighted a critical lesson: computing features across cities without verification can introduce meaningless signals. Visualizing the computed center revealed a point in Kansas, far from both LA and NYC, rendering the feature useless.

As a result, geographic features, including city-specific encodings, were removed. The final feature set prioritized **simplicity, interpretability, and cross-city transferability**.

Text feature approaches

We implemented a GenAI feature extractor that combined text embeddings and LLM-extracted scores. Listing descriptions were encoded using sentence-transformers and reduced via PCA (15 components), while LLM models (OpenAI or Anthropic) analyzed each listing's description, name, and amenities to produce numerical ratings (1–5) for luxury perception, cleanliness emphasis, host professionalism, location appeal, and amenities quality. The extractor included caching, error handling, and batch processing; embeddings were fit with PCA during training, whereas LLM features were generated directly and stored. Despite this, the high computational cost, latency, and limited predictive signal led us to abandon embeddings and LLM-derived scores in favor of simple keyword indicators (mentions_clean, mentions_luxury, mentions_4modern), which proved effective, interpretable, and transferable.

Final feature selection

After preprocessing, imputations, and feature engineering, the final dataset included **22 features**:

- **Original numerical and categorical features:** `accommodates`, `bathrooms`, `bedrooms`, `beds`, `room_ratio`, `host_response_rate`, `host_acceptance_rate`, `is_superhost`, `host_days_log`, `minimum_nights`, `instant_bookable`, `has_description`, `desc_length`, `has_host_about`, `response_speed`, `room_type`, `property_type`
- **Text-based features (keywords & metadata):** `mentions_clean`, `mentions_luxury`, `mentions_view`, `mentions_location`, `mentions_modern`, `has_neighborhood`, `name_length`

No city-specific features or geographic distances were retained, ensuring the model's features remain interpretable and transferable across cities.

---

4. Modeling (CRISP-DM Phase 4)

4.1 Models evaluated

Model	Outcome

XGBoost (all features)	Severe overfitting
Text embeddings	No improvement over baseline
Geographic clustering	Poor transfer to new city
Ridge regression	Best generalization

4.2 Why complex models failed

- High model capacity amplified noise
- Learned city-specific artifacts
- Cross-validation scores did not reflect real generalization

4.3 Final model choice

- **Ridge regression**
- Regularization parameter: **α = 10**
- Strong regularization to limit variance
- Explicit bias toward stable, transferable relationships

Several models were evaluated:

We initially trained XGBoost using all of the engineering features that were created during the exploratory data analysis phase. This included the aforementioned imputed values and log-transformed features, which, though statistically sound, may have reduced model generalizability as the model was severely overfit, and achieved an RMSE of only 0.44 on the test set which was worse than the baseline model.

In order to overcome this poor performance, we decided to explore text-based embeddings derived from listing descriptions which failed to improve over the baseline.

A geographical clustering feature was also tested by including a feature that represents distance from the city centre computed from mean longitude and latitude of the training set. However, as mentioned, this resulted in a “distance bug” causing a geographical reference point instead literally between los angeles and new york cities. Therefore, this introduced noise into our analysis and no useful added performance (RMSE = 0.3818).

After this, ridge regression modelling demonstrated a better performance (RMSE = 0.3816), demonstrated the strongest generalization when evaluated on held-out cities, outperforming more complex approaches despite its simplicity. This led to our final choice of model.

The final model choice

The final model was a Ridge regression with regularization strength α = 10, trained on a deliberately simplified and fixed set of 22 features. These features were selected because they represent stable signals that are meaningful across cities and do not depend on location-specifics. They include basic property characteristics (e.g. beds, bathrooms, accommodates), host-related features (e.g. host_response_rate, is_superhost,

host_days_log), and simple, interpretable text indicators capturing whether the listing mentions terms such as “clean”, “luxury”, or “modern”. All geographic features, city-specific encodings, and price-related variables were intentionally excluded to increase generalizability. – 6.4% better!

This combination proved most effective at generalization. Strong regularization to limit how complex the model could become limited variance, prevented overfitting, and biased the model toward stable, transferable relationships. In contrast to earlier complex approaches, this stripped-down model achieved the best performance on unseen cities, confirming that simplicity and controlled feature design were key to robust predictive performance.

---

5. Evaluation (CRISP-DM Phase 5)

5.1 Evaluation strategy

The model was trained on data from Los Angeles and New York City, validated on data from Chicago to evaluate its geographic generalization, and finally tested on data from Sydney. The primary evaluation metric used was Root Mean Squared Error (RMSE).

5.2 Results

Model	RMSE	Improvement
Dummy baseline	~0.406	—
Final model	**0.3799**	**~6–7%**

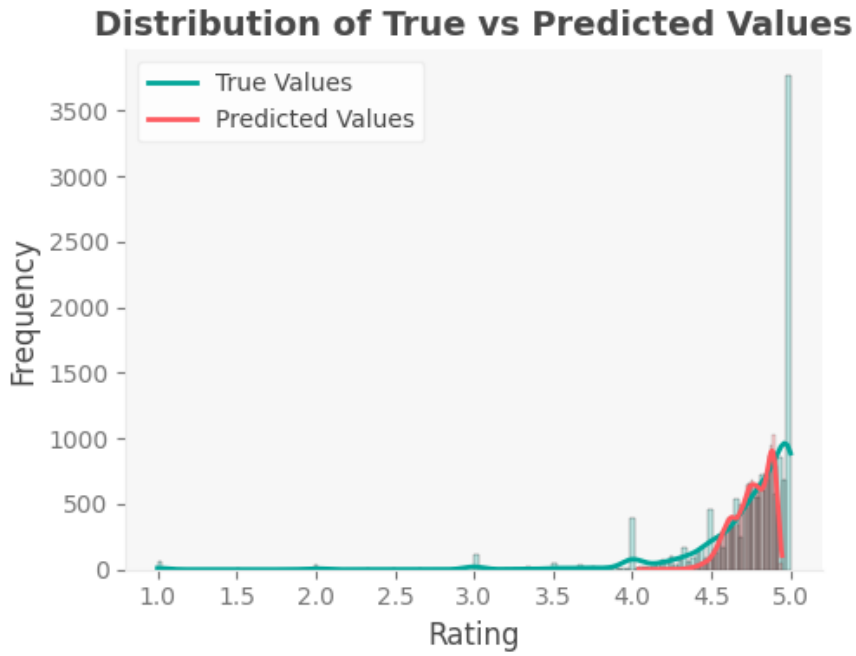
5.3 Interpretation

The model performed slightly better than the dummy baseline. Although the improvement is modest, one may argue that this improvement is meaningful, and that the compressed and skewed nature of the ratings distribution leaves only little room for improvement. However, to judge whether this argument holds, deeper error analysis is required.

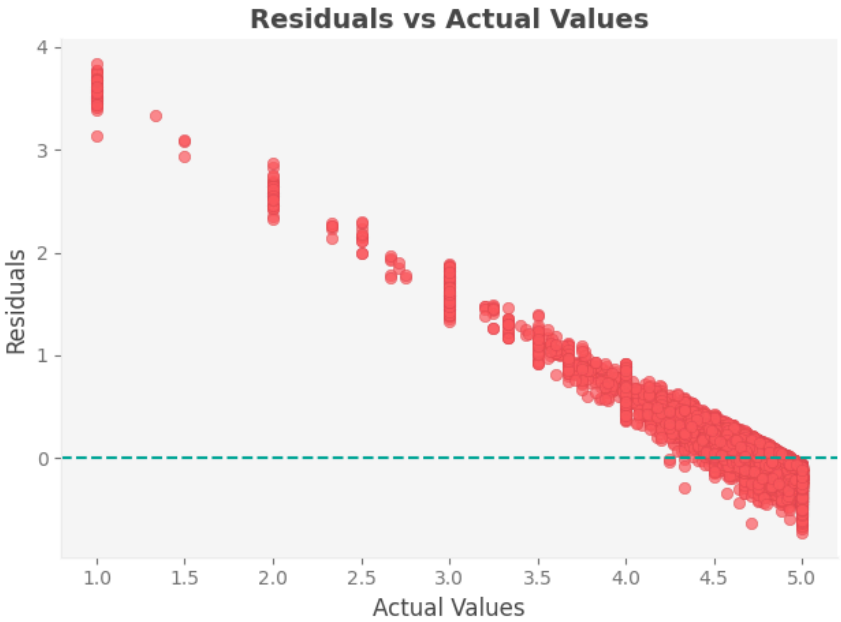
5.4 Error analysis (high level)

Error analysis consisted of the following examinations:

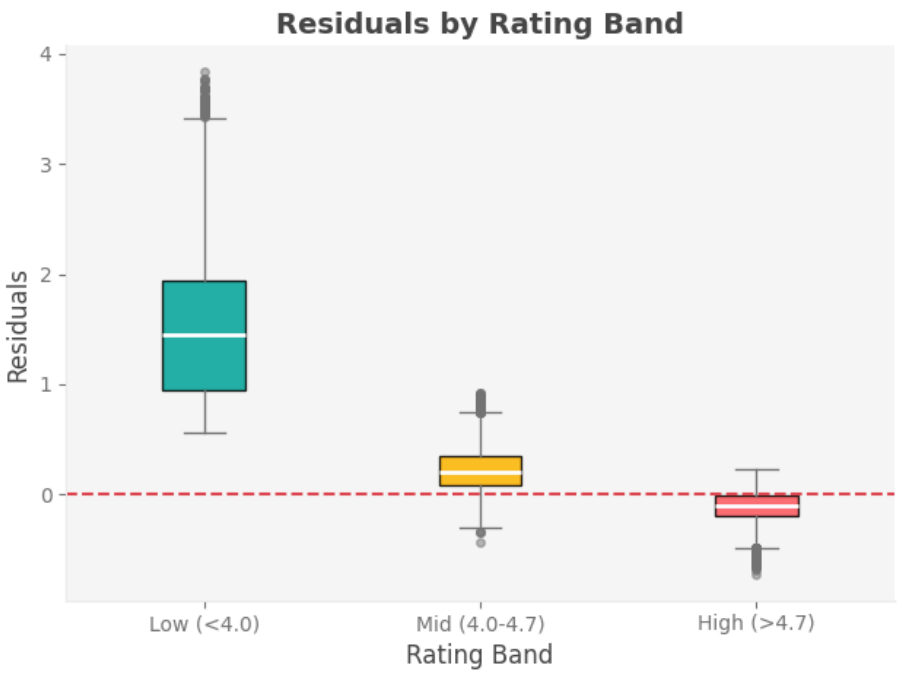
1. Prediction Distribution (True vs. Predicted): A comparison of the target values distributions revealed that the model failed in capturing the data’s true distribution.



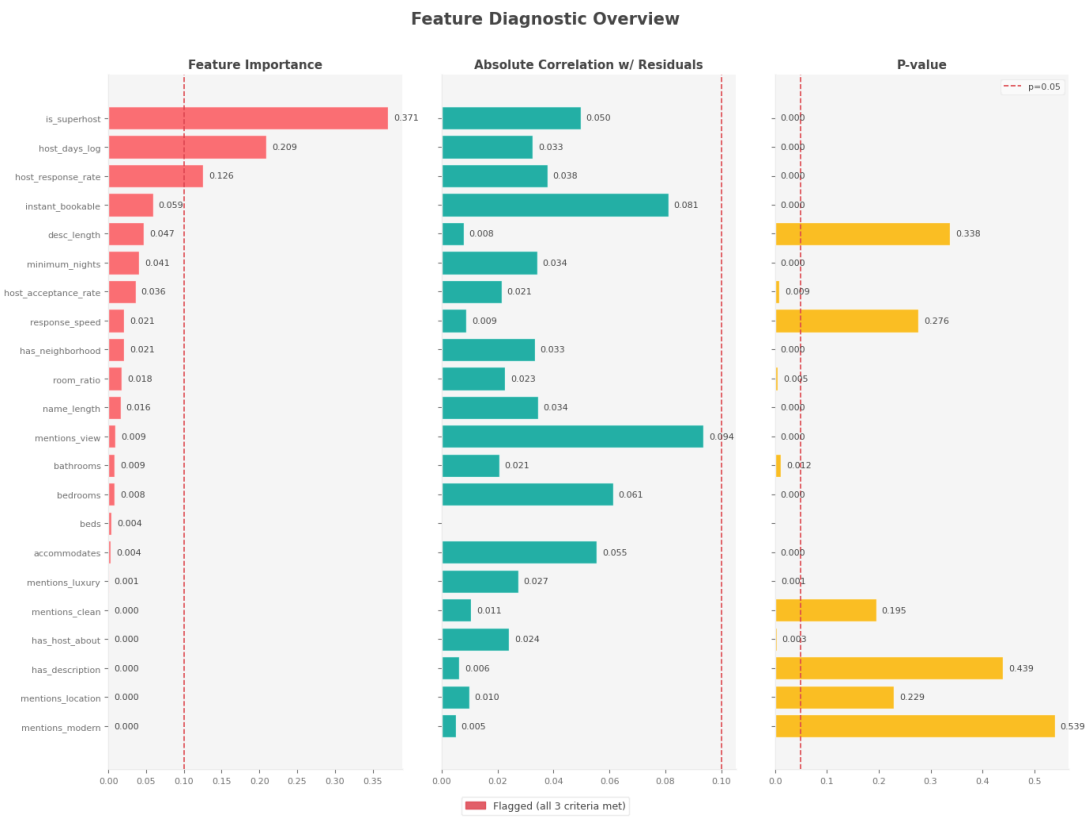
2. Systematic Bias (Residuals vs. True Ratings): We analyzed the relationship between the actual ratings and the model's residuals. The results show a near-perfect negative correlation (r = -0.96). Namely, the model systematically predicted the mean, resulting in higher scores for lower-rated listings, and lower scores for highly-rated ones.



3. Performance by Bins: The data was segmented into three tiers based on the true ratings (Low: <4, Mid: 4-4.7, High: >4.7). Then, we examined the residual distribution within each bin. The analysis confirmed that the error's magnitude and direction are strictly dictated by the true rating tier, with the model overpredicting the 'Low' bin, underpredicting the 'High' bin, and only performing well on the 'Mid' bin where true values lie close to the mean.



4. Feature-Level Error Analysis: We tried to identify individual features that may drive prediction errors. This was done by examining the correlation of each feature with the residuals, while also taking into account the feature's importance. All features were found to correlate weakly with the residuals (|r| < 0.1), regardless of their importance. In other words, no specific feature explains prediction errors in the test set.



In summary, the model acts primarily as a mean-predictor, that is, it minimizes the error by defaulting to the general mean rather than by capturing each feature's influence on the overall rating. Despite the fact that the attempt at improving the model by adding new engineered features did not bear fruit, it occurred to us only post-deployment that the core issue lies within the imbalance of the training data: low-rated listings are severely under-represented in them, resulting in the model not detecting the specific patterns that differentiate low-rated listings from average or above-average listings. The solution to this problem is to over-sample low-rated listings during training in order to enhance their signal and better capture the variance in the data.

---

6. Deployment (CRISP-DM Phase 6)

6.1 Deployed system

The model was deployed as an interactive Streamlit application, hosted on a Nebius virtual machine. The application accepts the data as a CSV file and generates predictions alongside summary statistics. It also automatically compares the model's RMSE to that of a baseline model.

6.2 Production considerations

The chosen model was saved as a .pkl file. The user's input undergoes the same preprocessing steps as those applied to the training data. The system is highly flexible due to the use of configurable paths and parameters, and it is designed with modularity in mind, in a way that allows future extensions as well as retraining.

---

7. Lessons Learned

7.1 Technical lessons

One of the clearest technical takeaways from this project is that model complexity does not reliably translate into better generalization, particularly on tabular data with a compressed target distribution. More expressive models such as XGBoost had sufficient capacity to fit city-specific noise in the training data, which severely undermined their ability to transfer to an unseen city. Ridge regression, by contrast, imposed strong regularization that constrained the model to learn only the most stable and broadly applicable signals, ultimately outperforming its more complex counterparts on the Chicago test set. This reinforces the general principle that regularization is not merely a safeguard against overfitting but a deliberate design choice that shapes what a model is able to learn.

A second critical technical lesson concerns the reliability of cross-validation as an evaluation tool. Throughout the project, cross-validation scores on the training cities (Los Angeles and New York City) did not faithfully predict how models would perform on Chicago. This discrepancy arises because cross-validation, when applied within the same set of cities,

cannot detect city-specific artifacts learned by the model. Only a true geographical holdout exposes these generalization failures, making it essential to evaluate on entirely unseen locations early and repeatedly during development rather than as a final checkpoint.

The distance-to-center bug offered a vivid demonstration of how a feature can appear structurally valid while being substantively meaningless. Because the geographic center was naively computed across both training cities simultaneously, the resulting reference point fell in Kansas, producing a distance metric that encoded no useful information about either city. The bug went undetected until features were visualized directly. This underscores that feature engineering must be paired with systematic validation and visual inspection; trusting a feature's construction logic without verifying its empirical distribution is a significant and avoidable risk.

7.2 Process lessons

From a process perspective, the most consequential lesson was the importance of testing geographic generalization at the outset of model development rather than treating it as a final evaluation. In this project, multiple modeling approaches were refined and discarded based on cross-city performance, a feedback loop that would have been far shorter and less costly had a city holdout been integrated from the beginning. Establishing a held-out city as a fixed validation benchmark early on would have directed attention toward transferable features and simple models much sooner, reducing wasted iteration on complex approaches that were fundamentally ill-suited to the task.

Starting with an interpretable baseline also proved to be a sound strategy. The dummy mean-predictor, with an RMSE of approximately 0.406, immediately framed the difficulty of the task and provided a stable reference point against which all subsequent models were judged. Without a clear baseline, it would have been easy to mistake minor RMSE fluctuations for meaningful progress. Similarly, maintaining a deliberate record of failed experiments, including XGBoost overfitting, embedding approaches that added no signal, and the geographic clustering that introduced noise, allowed the team to reason systematically about what was and was not working rather than revisiting the same dead ends.

Finally, the costs of premature complexity were demonstrated repeatedly throughout the project. The early pursuit of rich text features through LLM-based extraction and sentence embeddings consumed significant computational resources and development time before producing any measurable gain over the baseline. A better approach would have begun with simple keyword indicators and only escalated to more complex representations if those indicators proved clearly insufficient. When the problem is already constrained by a compressed target distribution, complexity rarely compensates for a fundamental ceiling on predictive signal, and restraint in model design is often the most effective strategy available.

Finally, by not addressing the skewed nature of the data in the training phase, the model was effectively penalized for learning anything beyond the mean. With approximately 95% of ratings concentrated between 4.5 and 5.0, low-rated listings were so severely under-represented that the model had insufficient signal to learn what differentiates them

from average or high-rated ones. As a result, the model defaulted to mean prediction, a behavior confirmed by the near-perfect negative correlation ($r = -0.96$) between true ratings and residuals observed during error analysis. Techniques such as over-sampling low-rated listings, applying instance weights, or constructing a more balanced training set should have been considered before tuning model architecture, since no amount of model complexity can compensate for a training distribution that obscures the very variance the model is asked to predict.

---

8. Future Work

- Earlier city holdout during development
- More careful budgeting for LLM feature extraction
- Explore ordinal regression
- Add explainability outputs for hosts
- Automate retraining and monitoring

Check:

Future work can improve both the technical robustness of the model and its business impact for hosts and the Airbnb platform.

From a modeling perspective, earlier use of city-level holdout validation during development would help detect generalization issues sooner and reduce time spent on models that do not transfer across markets. Alternative targets such as ordinal regression could better reflect the ordered nature of review scores. In addition, while large language models showed limited value in this project, future work could explore more targeted and budget-aware LLM usage, for example extracting structured signals only for new or atypical listings. Finally, adding model explainability outputs (e.g., per-listing feature contributions) and automating model retraining and performance monitoring would support reliable long-term deployment.

From a business perspective, future iterations should focus more explicitly on actionable levers under host control. The model highlights that hosts benefit from strong responsiveness, clear and informative descriptions, and signals of professionalism such as superhost status. Airbnb could use these insights to provide guided feedback or training, for example encouraging hosts to improve response rates, optimize property descriptions to highlight desirable attributes (e.g., cleanliness, modernity, luxury), or better configure their space (e.g., converting rooms into bedrooms where feasible). By aligning predictive modeling with host-facing recommendations, the system could help improve listing quality, guest satisfaction, and ultimately reinforce Airbnb's credibility and trust as a marketplace.

---

9. Conclusion

This project demonstrates that **careful feature selection, strong regularization, and disciplined evaluation** can outperform complex models in low-variance real-world problems.
By prioritizing generalization and deployment readiness, we built a practical and defensible rating prediction service.

Check:

This project shows that in a low-variance, real-world prediction task such as Airbnb review scores, robust generalization and disciplined modeling are more valuable than complex techniques. Although advanced approaches including XGBoost, geographic features, text embeddings, and LLM-derived signals were explored, they consistently failed to transfer across cities and often underperformed a simple baseline due to overfitting and sensitivity to noise. A deliberately constrained Ridge regression model with strong regularization ($\alpha = 10$), trained on a small, fixed set of interpretable and city-agnostic features, achieved the best performance on an unseen city, demonstrating that simplicity and regularization are critical in this setting. The project also highlighted key production lessons: feature bugs can silently destroy performance if not validated, preprocessing must be identical at training and inference, and cross-validation alone is insufficient to guarantee real-world generalization. Deployed as a Streamlit application on a Nebius VM, the final system provides a practical and defensible prediction service, while emphasizing that host-controllable factors such as responsiveness, professionalism signals, and clear descriptions are stable drivers of ratings and offer actionable value despite the strong ceiling effect inherent in the target variable.

Link to repo: https://github.com/RuthAdler/airbnb-rating-prediction.git